

Exception Handling in C++

1. Conceptual Foundation

- Exception handling in **C++** serves the same *logical purpose* as in Java: to handle runtime anomalies and prevent abrupt program termination.
 - However, **C++ is not purely object-oriented in its exception mechanism**. It allows **throwing and catching both primitive and user-defined types**, unlike Java (which enforces class-based exceptions derived from Throwable).
-

2. Throwing Mechanism

C++ allows throwing:

- **Primitive types**
 - `throw 10;` // int
 - `throw 'x';` // char
 - `throw 3.14;` // double
 - `throw "error msg";` // const char*
- **Objects (user-defined types)**
 - `MyException e;`
 - `throw e;`

Key Note

- throw **copies** the object (by value) at the time of throw, not by reference. Therefore, **copy constructors** and **destructors** play a critical role.
-

3. Catching Mechanism

- Catch blocks are matched **based on type compatibility**, similar to Java's class hierarchy but determined **at compile-time**.

Syntax Examples

```
catch (int x)           // catches integer exceptions
catch (const char* msg) // catches string exceptions
catch (MyException& obj) // catches user-defined exceptions
catch (...)             // generic catch (matches all types)
```

Important Rule

- **Specific catches must precede generic catch (catch(...))**, else the specific ones become unreachable (similar to Java's order of catch blocks).
-

4. Generic Catch Block

- Represented as:

```
catch (...) { // handles any type of exception not matched earlier }
```

- Equivalent to Java's catch (Exception e) but **without knowing the type**.

5. Exception Propagation

- If an exception is not caught within the same function, it **propagates up the call stack**.
- If no function in the call hierarchy catches it, the program **terminates** by calling:
- `std::terminate()`
- Unlike Java, **C++ does not have a built-in stack trace mechanism** unless manually implemented or using libraries like `<exception>` or `<stdexcept>`.

6. Stack Unwinding

- **When an exception is thrown**, all **objects created since the beginning of the current try block** are destroyed in **reverse order** of their creation.
- Destructors are automatically invoked for these objects before control transfers to the appropriate catch block.

Example

```
try
{
    MyClass obj1;
    MyClass obj2;
    throw 10;
}
catch (int e)
{
    cout << "Caught: " << e;
}
```

Order of destruction:

obj2 → obj1 → then control goes to catch.

Equivalent Concept in Java

- Behaves like a **finally block** but automatic through **RAII (Resource Acquisition Is Initialization)** principle.

7. The std::exception Hierarchy

```
exception
├── logic_error
│   ├── domain_error
│   ├── invalid_argument
│   ├── length_error
│   └── out_of_range
├── runtime_error
│   ├── range_error
│   ├── overflow_error
│   └── underflow_error
```

- **logic_error**: Problems in the program's logic (e.g., invalid index).
 - **runtime_error**: Problems during execution (e.g., overflow, allocation failure).
-

RTTI (Run-Time Type Identification)

1. RTTI (Run-Time Type Identification)

Definition

- RTTI (Run-Time Type Identification) is a mechanism in **C++** that allows the **type of an object to be determined at runtime**.
 - It allows programs to **safely identify or convert polymorphic objects** during execution.
 - It is required in **polymorphic languages** (C++, Java, C#) because object type is not always known at compile time due to **inheritance** and **upcasting**.
 - **Not required in non-polymorphic languages** (like C) — because object type is always known at compile time.
-

2. Need for RTTI

- Used when a **base class pointer/reference** is pointing to a **derived class object** and we want to check **which derived type** it actually refers to at runtime.
- Helps in **type-safe downcasting** and **object identification**.

Example scenario:

```
Shape* s;           // base pointer  
s = new Circle();  // derived object
```

- At compile time, compiler only knows s is of type Shape*, but **not** which derived type (Circle, Triangle, etc.) it actually points to.
 - RTTI allows checking the **actual derived type** of an object at runtime..
-

3. How RTTI is implemented

RTTI in C++ can be achieved using:

1. **typeid** operator
2. **dynamic_cast** operator

Header required:

```
#include <typeinfo>
```

4. typeid Operator

Syntax

```
typeid(object).name(): //name() is the method of typeid class which return the name of runtime type
```

Returns

- A reference to an object of class **type_info** that contains information about the type of the expression passed.
- In case of **non polymorphic** class **typeid** checks the type of pointer or reference (class doesn't contain any virtual function).
- IF class is polymorphic then it checks the content.

type_info Class

The type_info class describes the type information generated by the compiler.

Method	Description
bool operator==(const type_info&)	Checks whether two types are the same
bool operator!=(const type_info&)	Checks whether two types are different
const char* name()	Returns implementation-defined type name

Example (Basic Types)

```
int i; float f; char* p; int* ptr;
cout << typeid(i).name(); //int
cout << typeid(f).name(); //float
cout << typeid(p).name(); //char*
cout << typeid(ptr).name(); //int*
```

Example (User-Defined Types)

```
class MyClass1 {};
class MyClass2 {};
MyClass1 m1;
MyClass2 m2;
cout << typeid(m1).name();
cout << typeid(m2).name();
```

Example (Polymorphic)

```
class Base { virtual void show() {} };
class Derived : public Base {};
Base* b = new Derived;
cout << typeid(*b).name(); // prints Derived
```

If the base class is **non-polymorphic** (i.e., no virtual function), then:

```
cout << typeid(*b).name(); // prints Base
```

Thus, typeid behaves differently for polymorphic and non-polymorphic types.

5. dynamic_cast Operator

Used to **safely convert (downcast)** one polymorphic type to another at runtime.

Syntax

```
dynamic_cast<target_type*>(expression)
dynamic_cast<target_type&>(expression)
```

Conditions

1. The base class must be **polymorphic** (i.e., must contain at least one virtual or pure virtual function).
2. The cast must be between **pointers or references** only.
3. dynamic_cast uses the **vtable** to check actual type at runtime.

Behavior

- If cast **succeeds**:
 - Returns valid converted pointer/reference.

- If cast **fails**:
 - Returns **NULL** for pointers.
 - Throws **bad_cast exception** for references if reference return NULL.

Example (Pointer Downcasting)

```
class Base { virtual void show() {} };
class Derived : public Base { void print() {} };

Base* b = new Derived;
Derived* d = dynamic_cast<Derived*>(b);
if (d)
{
    d->print();    // success
}
else
    cout << "Cast failed";
```

Example (Reference Downcasting)

```
try {
    Derived& ref = dynamic_cast<Derived&>(baseRef);    //if casting returns the Null it throws the exception
}
catch (bad_cast& e) {
    cout << e.what();    // prints "bad_cast"
}
```

6. Why Downcast?

- Upcasting is done to **achieve polymorphism (late binding)** — we use base pointer to refer to derived object.
- **Downcasting** is needed when:
 - We want to access **derived-specific members** (which are not in the base class).
 - We need to check whether the base pointer actually holds a particular derived object.

Example

```
void caller(Course* ptr) {
    ptr->start();    // polymorphic
    Dac* d = dynamic_cast<Dac*>(ptr);
    if (d) {
        d->orientation();    // derived-specific call
    }
}
```

7. Drawback of Normal Typecasting

Using simple cast:

```
Derived* d = (Derived*)basePtr;
```

Drawbacks:

- It **does not** check vtable information or actual runtime type.
- Can lead to **undefined behavior** if basePtr does not actually point to a Derived object.
- It **forces** conversion, even if incorrect — can cause undefined behavior or crash.
- No runtime safety → program crash possible.

Hence, **prefer dynamic_cast** for type-safe downcasting.

8. RTTI with References

When **using references** instead of pointers:

```
try {
    Derived& d = dynamic_cast<Derived&>(baseRef);
}
catch (bad_cast& e) {
    cout << e.what(); // exception if cast fails
}
```

- Works only for polymorphic base classes.
 - `dynamic_cast` throws `bad_cast` when conversion fails (for references).
 - Safe and exception-handled approach.
-

9. typeid vs dynamic_cast

Feature	typeid	dynamic_cast
Purpose	To identify actual object type	To safely cast to derived type
Works with	Any type (even non-polymorphic)	Only polymorphic types
Uses vtable	No	Yes
Checks inheritance hierarchy	No	Yes
Return Type	Returns reference to <code>type_info</code>	Returns converted pointer/ref
Failure Behavior	Not applicable	Returns NULL (pointer) or throws <code>bad_cast</code> (reference)
Typical Use	For comparing or printing type	For type-safe downcasting
Requirement	<code>#include <typeinfo></code>	Polymorphic base class
Runtime check	Partial	Complete

11. Example Snippets (Conceptual)

Using `typeid`:

```
if (typeid(*ptr) == typeid(Dac))
{
    Dac* d = (Dac*)ptr; // manual cast after confirmation
    d->orientation();
}
```

Using `dynamic_cast`:

```
Dac* d = dynamic_cast<Dac*>(ptr);
if (d)
    d->orientation();
```

With Reference:

```
try {
    Dac& ref = dynamic_cast<Dac&>(obj);
}
catch (bad_cast& e) {
```

```
    cout << e.what();  
}
```

12. Internal Working

- **typeid** only checks the immediate type; it does not use the vtable. Therefore, it does not require polymorphism.
 - **dynamic_cast** uses **vtable** entries to verify entire inheritance hierarchy, hence polymorphism is mandatory.
-

11. One.cpp Concept Summary

```
Shape* s = new Circle();    // Upcasting (late binding)  
Circle* c = dynamic_cast<Circle*>(s); // Safe downcasting
```

Why Upcast?

- To achieve **runtime polymorphism** and maintain flexibility.

Why Downcast?

- To access **derived-specific behavior** safely.
-

const_cast

Purpose:

Used to **add or remove const or volatile qualifiers** from a variable.

Syntax:

```
const_cast<type>(expression)
```

Example:

```
void printAndModify(const int* ptr) {  
    int* modifiable = const_cast<int*>(ptr);  
    *modifiable = 20; // Safe only if original variable is NOT actually const  
}
```

Key Points:

- Allows modification of a value that was **passed as const**, but **only if** the original data was not declared const.
- If you try to modify a **truly const variable**, it results in **undefined behavior**.
- Commonly used to call **non-const functions** with **const arguments**.

Warning:

Use only when you are absolutely sure the original data is not truly constant.

static_cast

Purpose:

Performs **compile-time, non-polymorphic type conversion**.

Syntax:

```
static_cast<type>(expression)
```

Examples:

```
double d = 4.5;
int i = static_cast<int>(d); // Safe numeric conversion

base* b = new sub1;
sub1* s = static_cast<sub1*>(b); // Downcast - works, but unsafe if 'b' not really 'sub1'
```

Key Points:

- Used for **standard conversions** (e.g., int ↔ float, pointer ↔ void*, base ↔ derived).
- **Compile-time checked** — if types are incompatible, compiler error occurs.
- Does **not** check **runtime type information (no vtable check)**.
- Suitable for **upcasting** (derived → base), but **unsafe for downcasting** (base → derived).

Summary:

- Type-safe at compile time
- No runtime safety
- Should not be used for polymorphic downcasts

reinterpret_cast (for completeness)

Purpose:

Performs **bitwise-level reinterpretation** of data. Used for low-level operations like pointer-to-int or unrelated pointer conversions.

Syntax:

```
reinterpret_cast<type>(expression)
```

Example:

```
int* p = &x;
char* c = reinterpret_cast<char*>(p);
```

Key Points:

- Reinterprets bits without changing actual data.
 - **No type safety** — use with extreme caution.
 - Useful in low-level system code or hardware interfacing.
-

Co-variant Return Type in C++

Definition

- A *co-variant return type* in C++ allows a derived class to override a virtual function in the base class and return a **more specific type** (derived type) than the one returned by the base class function.
 - This applies only to return types that are **pointers or references** to classes within the same inheritance hierarchy.
 - It preserves polymorphism, enhances type safety, and provides greater clarity to the programmer regarding what object type is actually being returned.
-

Example Concept

```
class Base {
public:
    virtual A* disp() {
        return new A();
    }
};

class Derived : public Base {
public:
    B* disp() { // Allowed because B is derived from A
        return new B();
    }
};
```

In the above example, `disp()` in the base class returns an `A*`, while the overridden version in the derived class returns a `B*`. This is valid because `B` is a subclass of `A`.

Conditions for Co-variant Return Type

1. The function in the base class must be declared as **virtual**.
 2. The overriding function must have the **same function signature** except for the return type.
 3. The return types must be **pointers or references** to classes.
 4. The return type in the derived class must be a **derived type** of the return type in the base class.
-

Reason for Allowing Co-variant Return Type

Co-variant return types enhance **type safety** and **code clarity** in polymorphic class hierarchies.

They allow a derived class to provide a more specific return type, eliminating the need for explicit type casting when using overridden methods.

Example:

```
Base* b = new Derived();
B* obj = b->disp(); // Safe and clear, returns B* directly
```

This makes the interface of the derived class more descriptive and aligned with its behavior.

Invalid Case and Its Reason

If a base class function returns a **derived type**, and the overriding function in the subclass tries to return a **base type**, it is not allowed.

Example:

```
class Base {
public:
    virtual B* disp() {
        return new B();
    }
};
class Derived : public Base {
public:
    A* disp() {    // Not allowed
        return new A();
    }
};
```

If this were allowed, a statement like:

```
Base* ptr = new Derived();
B* ref = ptr->disp();
```

would cause a problem at runtime because the compiler assumes that `disp()` returns a `B*`, but it would actually return an `A*`, leading to type mismatch.

To prevent such unsafe situations, the compiler disallows overriding a virtual function with a **less derived** return type.

Practical Advantage

Although both of the following are syntactically valid:

```
FourWheeler* disp();
Car* disp();
```

only the second prototype (`Car* disp()`) conveys to the user exactly what type of object is being returned.

Hence, co-variant return types make class interfaces more informative and improve the readability and predictability of the code.

Key Points Summary

Concept	Description
Definition	Allows overriding functions to return a more derived type than the base function.
Requirement	Base function must be virtual and return a pointer or reference.
Allowed	Base returns <code>Parent*</code> or <code>Parent&</code> , derived returns <code>Child*</code> or <code>Child&</code> .
Not Allowed	Base returns <code>Child*</code> , derived returns <code>Parent*</code> .
Reason for Disallowance	Violates type safety; leads to incorrect dynamic binding.
Advantage	Improves readability, avoids unnecessary type casting, supports clear polymorphism.

